Universidad Nacional de Córdoba
Facultad de Ciencias Exactas, Físicas y Naturales
Ingeniería en Computación

Redes de Computadoras

Monografía

MQTT y Protocol Buffers

Un stack de comunicación en tiempo real para IoT, robótica y mensajería con servicios en la nube

Equipo
WireGuardians

Viberti, Benjamin[1]	DNI 46224179
Espinoza Sutta, Aaron Alejandro[1]	DNI 96009173
Cleri, Juan Ignacio[1]	DNI 46452662
Pineda, Juan Ignacio[1]	DNI 45591343
Grafión, Atilio Leonel[1]	DNI 43940195
Badenes, Tomás[1]	DNI 44785038
Oviedo, Ignacio Nicolás[1]	DNI 44169239
Mendez, Jorge Nicolas[1]	DNI 41301342

Docente
Oliva Cuneo, Facundo Nicolas

[1]{b.viberti,aaron.espinoza_4500,ignacio.cleri,juan.ignacio.pineda,atilio.grafion,tomasbadenes,ignacio.oviedo.239,jorge.mendez}@mi.unc.edu.ar

BORRADOR

24 de octubre de 2026

Índice

Introducción **3**
1. Objetivos 4
2. Alcance 4
Marco teórico **5**
1. Modelo publicador/suscriptor 5
2. MQTT 5
3. Protocol Buffers 5
4. Stack MQTT y Protocol Buffers 5
5. Comparación con alternativas 5
   5.1. MQTT, HTTP y CoAP 5
   5.2. Protocol Buffers y JSON 5
Desarrollo **6**
1. Diseño experimental 6
   1.1. Plataforma común 6
   1.2. Experimento 1: sincronización de luciérnagas 6
   1.3. Experimento 2: MQTT y HTTPS para telemetría 9
Conclusiones **14**
Bibliografía **15**
Anexos **16**
A. Mensajes del experimento de sincronización 16

BORRADOR

Introducción

Gran parte de la Internet comercial se apoya en el modelo de solicitud y respuesta de HTTP: un cliente consulta a un servidor cuando necesita un dato y recibe la respuesta, que suele venir codificada en formatos de texto como HTML, JSON o XML. Este modelo está pensado para intercambios puntuales, que el cliente inicia a demanda, y no para flujos de datos continuos ni para que el servidor tome la iniciativa.

Un número creciente de sistemas no encaja en ese modelo. Los dispositivos del Internet de las Cosas, las flotas de robots y las aplicaciones de mensajería móvil generan flujos continuos de telemetría, eventos y comandos a alta frecuencia y en ambos sentidos, desde equipos de recursos acotados que operan sobre redes celulares o inalámbricas inestables y detrás de firewalls fuera del control de quien administra el servidor. Esta combinación plantea dos limitaciones de fondo para la ingeniería de redes: el costo, en ancho de banda y energía, de consultar periódicamente y de codificar los datos en formatos de texto; y la imposibilidad de que un servidor inicie comunicaciones hacia dispositivos que no admiten conexiones entrantes.

Frente a este panorama, muchos de estos sistemas han recurrido a la mensajería de tipo publicador/suscriptor, que desacopla a los participantes mediante un intermediario. En este campo se inscribe la combinación de MQTT y Protocol Buffers, un stack pensado para la comunicación en tiempo real con ancho de banda limitado. MQTT es un protocolo liviano de transporte de mensajes, organizado en torno a un broker, concebido para entornos restringidos (como los de comunicación máquina a máquina y de Internet de las Cosas) en los que se necesita una huella de código reducida o el ancho de banda es escaso. Protocol Buffers es un mecanismo, independiente del lenguaje y de la plataforma, para serializar datos estructurados y tipados en un formato más compacto y rápido que los textuales, a partir de un esquema compartido.

Su adopción en sistemas reales es amplia y heterogénea. En la mensajería a gran escala, Facebook recurrió a MQTT en Messenger para sostener una conexión persistente con sus servidores sin agotar la batería de los teléfonos, lo que permitió, en muchos casos, entregar los mensajes en cientos de milisegundos en lugar de varios segundos. En robótica, InOrbit, una empresa que ofrece una plataforma de orquestación de flotas de robots, implementa la comunicación entre servicios e integraciones desplegadas *on the edge* utilizando MQTT y mensajes definidos con Protocol Buffers, donde brokers desplegados en la nube reciben datos en tiempo real de miles de robots y dispositivos IoT, que a su vez son controlados por el mismo canal, aun en instalaciones con redes inestables. Desarrollado por la misma empresa, el producto de código abierto OpenRobOps libera las definiciones de su protocolo de comunicación en estructuras Protobuf que fijan el formato de las comunicaciones entre robot y plataforma.

Un rasgo de este esquema es especialmente relevante desde el encaminamiento: el dispositivo abre una conexión persistente hacia el broker alojado en la nube, cuya dirección suele estar habilitada en el firewall, y por esa misma conexión el broker le entrega los mensajes, sin necesidad de conexiones entrantes. El principio rector del stack es, entonces, trasladar la complejidad desde la red hacia el protocolo de aplicación: ningún participante necesita conocer la ubicación de los demás, sino solo la del broker, y el contrato sobre el formato de los datos se fija una única vez en un esquema distribuido entre los clientes. De este modo, la comunicación en tiempo real entre miles de entidades heterogéneas resulta viable sobre redes que no fueron concebidas para ese fin.

BORRADOR

1. Objetivos

Se desea realizar un análisis riguroso del stack de comunicación formado por MQTT y Protocol Buffers, como alternativa para el intercambio de datos en tiempo real, a alta frecuencia y con ancho de banda limitado, en aplicaciones de Internet de las Cosas, robótica y mensajería. El estudio sigue el recorrido de un mensaje desde el dispositivo que lo genera hasta el broker que lo distribuye, y evalúa cómo cada capa de la arquitectura contribuye a que ese recorrido sea eficiente y confiable.

Más que catalogar las prestaciones de ambas tecnologías, el trabajo busca explicar por qué resultan adecuadas para el problema planteado: sostener una comunicación continua sobre redes que el diseñador no controla, con enlaces inestables, firewalls de terceros y trayectos que atraviesan múltiples sistemas autónomos. Con ese fin, el análisis se articula en torno a dos ejes centrales de la materia, que pertenecen a capas distintas: la capa de aplicación (capa 7), donde operan MQTT y Protocol Buffers, y la capa de red (capa 3), donde el ruteo determina por qué camino llegan los mensajes al broker y cómo se restablece ese camino cuando la red cambia.

Para alcanzar este propósito se plantean los siguientes objetivos específicos:

- Presentar el marco teórico del modelo publicador/suscriptor y describir cómo lo implementa MQTT (broker, tópicos, niveles de calidad de servicio, sesiones y seguridad), junto con el mecanismo de serialización de Protocol Buffers, comparándolos con alternativas como HTTP, CoAP y JSON.
- Relacionar el funcionamiento de MQTT con la capa de transporte, sobre la que opera mediante TCP (con keep-alive y TLS), y distinguir las garantías de entrega de sus niveles de calidad de servicio de la confiabilidad que ya ofrece TCP.
- Analizar el recorrido de los mensajes en la capa de red (capa 3): direccionamiento, NAT y firewalls, y el efecto de la convergencia de los protocolos de ruteo interno (OSPF, con RIP y EIGRP como comparación) y externo (BGP) sobre la disponibilidad de las conexiones TCP de larga duración en las que se apoya MQTT.
- Contrastar el ruteo por destino IP de la capa de red con el ruteo por tópicos que realiza el broker en la capa de aplicación.
- Examinar casos de uso reales en robótica, mensajería e Internet de las Cosas, como los de InOrbit, para determinar las ventajas y limitaciones del stack en entornos de alta frecuencia y recursos acotados.

2. Alcance

El análisis se centra en MQTT y Protocol Buffers como protocolos de la capa de aplicación, y en su relación con las capas de transporte y de red. HTTP, CoAP y JSON aparecen como puntos de comparación, no como objetos de estudio en sí mismos.

La parte experimental se realiza sobre placas ESP32 emuladas con QEMU y un broker Mosquitto, sin hardware físico. Los resultados describen, por lo tanto, el comportamiento de los protocolos sobre una red Ethernet emulada, con pérdida y latencia controladas, y no el de un enlace inalámbrico real.

BORRADOR

Marco teórico

1. Modelo publicador/suscriptor

2. MQTT

3. Protocol Buffers

4. Stack MQTT y Protocol Buffers

5. Comparación con alternativas

5.1. MQTT, HTTP y CoAP

5.2. Protocol Buffers y JSON

BORRADOR

Desarrollo

1. Diseño experimental

Para observar el comportamiento del stack más allá del análisis teórico, se diseñaron dos experimentos. El primero pone a prueba MQTT y Protocol Buffers en un problema que se resuelve solo con intercambio de mensajes entre pares: la sincronización de osciladores, al estilo de las luciérnagas que terminan parpadeando al unísono. El segundo compara MQTT con HTTPS en una tarea que ambos resuelven de forma razonable, el envío periódico de telemetría a un servidor, y separa el efecto del transporte del efecto de la serialización.

Cada experimento se presenta en cuatro partes: el planteo (qué se quiere averiguar y por qué), la implementación, los resultados y su análisis.

1.1. Plataforma común

Los dos experimentos usan la misma plataforma: ocho placas ESP32 emuladas con QEMU, un broker Mosquitto y un backend que registra el tráfico y calcula las métricas.

La emulación impone, a cambio, algunas restricciones que valen para ambos experimentos:

- QEMU no emula WiFi. La red entre los nodos es Ethernet emulado o el stack de red de QEMU, de modo que el comportamiento propio de un enlace inalámbrico queda fuera del estudio.
- La emulación es por software y el timing no es idéntico al de un ESP32 físico. Por eso en cada corrida se mide el *factor de tiempo real*, el cociente entre el tiempo simulado y el tiempo del host.
- Las ocho instancias comparten la CPU del host, que limita la cantidad de nodos más que la memoria. Ese es el motivo del máximo de ocho nodos.

1.2. Experimento 1: sincronización de luciérnagas

1.2.1. Planteo

Cada ESP32 controla hasta cuatro LEDs, y cada LED es un oscilador que parpadea a su propio ritmo. Los LEDs se ajustan entre sí a partir de los mensajes MQTT que publican al disparar, con payload en Protocol Buffers, sin que ningún coordinador central calcule las fases. La pregunta de fondo es si el stack alcanza para que un conjunto de nodos llegue a parpadear al unísono, como lo hacen ciertas especies de luciérnagas, y a qué costo.

Motivación

El problema ejercita justamente los rasgos del stack que estudia la monografía. Cada disparo de un LED tiene que llegar, a través del broker, a todos los demás osciladores: es comunicación de muchos a muchos sobre el modelo publicador/suscriptor. Los mensajes son chicos y frecuentes, lo que pone en primer plano el tamaño de la cabecera MQTT y de la codificación de Protocol Buffers. Y la calidad de la sincronización depende de que esos mensajes lleguen, y lleguen a tiempo, de modo que la pérdida y la latencia de la red se traducen en un efecto observable.

Modelo de osciladores acoplados

Cada LED i tiene una fase θ_i que avanza con su frecuencia natural ω_i. Cuando la fase llega a 2π, el LED dispara, publica un mensaje de *flash* y reinicia su fase. Al recibir el flash de otro LED j, adelanta su fase en una cantidad proporcional al acoplamiento ε, y de ese modo tiende a alinearse con j.

BORRADOR

El grado de sincronización se mide con el parámetro de orden de Kuramoto:

$$r = \left| \frac{1}{M} \sum_{j=1}^{M} e^{i\theta_j} \right| \tag{1}$$

donde M es el número total de LEDs. Si las fases están dispersas, los términos se cancelan y r queda cerca de 0; si están alineadas, se suman y r se acerca a 1. Se considera que la red convergió cuando $r \geq 0{,}95$.

Cada LED es un oscilador independiente aunque comparta placa con otros. Con ocho ESP32 de cuatro LEDs hay hasta 32 osciladores.

Preguntas de investigación

1. ¿Pueden los nodos sincronizarse usando solo intercambio de mensajes entre pares por MQTT, sin un coordinador que calcule fases?
2. ¿Cómo afectan la cantidad de nodos, los LEDs por nodo, el acoplamiento, la pérdida, la latencia y el QoS a esa sincronización?
3. ¿Cuánto cuesta en mensajes, bytes y latencia la sincronización con Protocol Buffers sobre MQTT?

Hipótesis

- Con ε por encima de un umbral y sin pérdida, la red converge a $r \geq 0{,}95$.
- Aumentar la latencia retrasa la convergencia, pero no impide llegar a la sincronía.
- Con un 5 % de pérdida la convergencia se degrada, y QoS 1 debería compensar parte de esa pérdida a costa de más mensajes.
- La cantidad de mensajes por segundo crece de forma aproximadamente lineal con la cantidad de nodos, y el tiempo de convergencia depende de la topología y de ε.

1.2.2. Implementación

Entorno

El experimento corre sobre la plataforma común: ocho ESP32 emuladas con QEMU y un broker Mosquitto.

Arquitectura

Los nodos no se comunican entre sí de forma directa: todo lo que publican y reciben pasa por el broker (Figura 1). El backend observa el tráfico, calcula las métricas y envía la configuración de cada corrida; el frontend recibe los datos del backend por WebSocket y muestra en vivo el estado de cada LED, los flashes que circulan por la red y la evolución de las métricas.

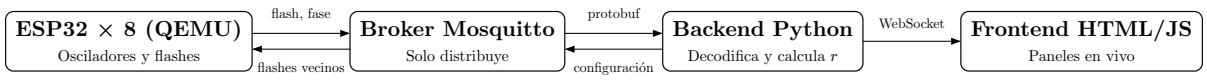


Figura 1: Arquitectura del experimento de sincronización: ocho nodos, un broker, el backend y el frontend.

Mensajes y tópicos

Los mensajes se definen en un único archivo `.proto`, compartido por el firmware y por el backend. El más frecuente es el flash, que cada LED publica al disparar:

BORRADOR

```
message Flash {
  uint32 esp_id = 1;
  uint32 led_id = 2;
  uint32 seq    = 3;
  uint64 t_us   = 4;
}
```

El archivo completo, con los mensajes de reporte de fase, de recepción y de configuración, está en el Anexo A. La Tabla 1 muestra qué mensaje viaja por cada tópico; en ellos, `{esp}` y `{led}` son números, de modo que `exp/esp/3/led/2/flash` es el disparo del LED 2 de la ESP 3.

Tabla 1: Tópicos del experimento de sincronización.

Tópico	Quién publica	Quién recibe	Mensaje
`exp/esp/{esp}/led/{led}/flash`	ESP	ESP vecinos, backend	`Flash`
`exp/esp/{esp}/led/{led}/phase`	ESP	backend	`PhaseReport`
`exp/esp/{esp}/rx`	ESP	backend	`RxEvent`
`exp/ctl/config`	backend	todos los ESP	`ExperimentConfig`

Solo el tópico `flash` forma parte del algoritmo. Los reportes de fase alimentan al frontend, y los mensajes de `rx` confirman cada recepción para medir la entrega; ambos son tráfico propio del experimento, que se mide como overhead y se reporta por separado del costo de la sincronización.

Firmware

Emulación de la red

Diseño de las corridas

Se varía un factor a la vez respecto de una configuración base: ocho ESP32 con cuatro LEDs cada una (32 osciladores), ε fijo, pérdida del 0 %, latencia de 0 ms y QoS 1. La Tabla 2 resume los factores y la pregunta que responde cada uno. Cada configuración se repite cinco veces con semillas fijas, y cada corrida dura 120 s o hasta que $r \geq 0{,}95$ se mantenga durante 30 s. Los mensajes, las métricas y la configuración usada se guardan en CSV o JSON.

Tabla 2: Factores del experimento de sincronización.

Factor	Valores a probar	Pregunta que responde
Nodos ESP	2, 4, 8	¿Cómo escala el tiempo de convergencia?
LEDs por ESP	1, 2, 4	¿Afecta la densidad de osciladores por nodo?
Acoplamiento ε	bajo, medio, alto	¿Hay un umbral para sincronizar?
Pérdida	0 %, 1 %, 5 %	¿Cuánto se degrada la convergencia?
Latencia	0 ms, 50 ms, 200 ms	¿Cómo afecta el tiempo de propagación?
QoS	0 y 1	¿Cuánto pesa la garantía de entrega?

Métricas

Cada métrica responde a una pregunta de la monografía (Tabla 3). Las de convergencia se calculan sobre $r(t)$; las de costo, sobre cada mensaje publicado.

BORRADOR

Tabla 3: Métricas del experimento de sincronización.

Métrica	**Cómo se calcula**	**Qué permite concluir**
Tiempo de convergencia	Primer instante con $r \geq 0{,}95$	Velocidad de sincronización
r final	Promedio de r en los últimos 10 s	Si la red queda sincronizada
Bytes por flash (Protobuf)	Tamaño del mensaje con cabeceras MQTT	Costo de ancho de banda
Bytes por flash (JSON)	Mismo mensaje en JSON	Ahorro de la serialización
Mensajes por segundo	Total de publicaciones / tiempo	Carga de la red
Tasa de entrega	Recibidos / enviados, por par emisor--receptor	Efecto de la pérdida y del QoS
Latencia flash → recepción	`RxEvent.t_us` − `Flash.t_us`	Tiempo real de propagación
Factor de tiempo real	Tiempo simulado / tiempo del host	Validez de la corrida

Las latencias se calculan con un reloj de referencia común, o con los relojes simulados sincronizados antes de cada corrida, para que la resta de marcas de tiempo de nodos distintos tenga sentido.

Validez
Si el factor de tiempo real cae por debajo de un umbral, la corrida se descarta, porque el timing de la emulación ya no representa al de un ESP32. Como el frontend, el backend y el broker comparten el host con las instancias, también compiten con ellas por la CPU.

1.2.3. Resultados

1.2.4. Análisis de resultados

1.3. Experimento 2: MQTT y HTTPS para telemetría

1.3.1. Planteo

Las mismas ocho ESP32 envían telemetría periódica a un servidor por dos caminos: MQTT a través del broker, y HTTPS con un POST por lectura. El contenido de cada lectura es idéntico en ambos caminos y se serializa en JSON o en Protocol Buffers, lo que permite separar dos efectos: el del transporte y el de la serialización.

A diferencia del experimento anterior, el caso de uso es neutral: enviar lecturas de los nodos al servidor es algo que ambas arquitecturas resuelven de forma razonable.

Motivación
El experimento responde a los objetivos que comparan MQTT con HTTP y que relacionan MQTT con la capa de transporte (TCP, keep-alive y TLS). Las dos variantes de HTTPS, las corridas con y sin TLS y las dos serializaciones permiten, además, separar cuánto de la diferencia se debe a la conexión persistente, cuánto a TLS y cuánto al formato del mensaje.

Fundamento
La diferencia principal entre ambos caminos no está en el payload, sino en cómo cada protocolo usa la conexión TCP (Tabla 4).

BORRADOR

Tabla 4: Diferencias entre MQTT y HTTPS en el uso de la conexión.

Aspecto	**MQTT**	**HTTPS**
Conexión	Una sesión TCP persistente por nodo	Una conexión por request, o reutilizada con keep-alive
Handshake TLS	Una vez, al conectar	En cada conexión nueva
Cabecera por mensaje	Mínimo 2 bytes, más el tópico	Headers HTTP en texto, varios cientos de bytes
Respuesta	PUBACK solo con QoS 1 o 2	Siempre una respuesta HTTP
Garantía de entrega	QoS 0, 1 o 2, definidos por el protocolo	Reintentos implementados por la aplicación
Iniciativa	El servidor puede enviar al nodo por la misma sesión	Solo el cliente inicia

Por eso se prueban dos variantes de HTTPS: una conexión nueva por request y una conexión reutilizada. La segunda es la comparación justa; la primera muestra el peor caso, el de los dispositivos que duermen entre envíos.

Preguntas de investigación

1. ¿Cuántos bytes cuesta enviar una lectura por MQTT y por HTTPS, contando cabeceras y handshakes?
2. ¿Cómo se comparan la latencia extremo a extremo y la tasa de entrega de ambos?
3. ¿Cómo se degradan ambos protocolos con pérdida y latencia en la red?
4. ¿Qué parte de la diferencia viene de la conexión persistente y qué parte de TLS?
5. ¿Cuánto cambia el costo al pasar de JSON a Protocol Buffers en cada transporte, y cómo depende ese ahorro del tamaño y la forma del mensaje?

Hipótesis

- MQTT envía menos bytes por lectura que HTTPS, y la diferencia crece cuando el período de envío es corto.
- HTTPS con conexión reutilizada reduce gran parte de la diferencia, pero los headers HTTP siguen pesando más que la cabecera MQTT.
- HTTPS con una conexión por request es el caso más costoso, sobre todo con TLS y latencia alta, porque cada handshake suma idas y vueltas.
- Con pérdida, MQTT QoS 1 mantiene la entrega a costa de duplicados, mientras que MQTT QoS 0 pierde lecturas.
- Protobuf reduce el payload en ambos caminos, pero su efecto sobre el total es mayor en MQTT, donde el payload es la mayor parte del mensaje. En HTTPS, los headers diluyen el ahorro.
- El ahorro de Protobuf crece con la cantidad de valores numéricos, porque JSON repite los nombres de campo y escribe los números como texto. Con texto corto, la diferencia debería ser menor.
- El tiempo de serialización en el nodo es menor con Protobuf (nanopb) que con una librería JSON, aunque la diferencia es chica frente a la latencia de red.

1.3.2. Implementación

Entorno

El experimento corre sobre la plataforma común: ocho ESP32 emuladas con QEMU y un broker Mosquitto.

BORRADOR

Arquitectura

En cada corrida se usa un solo camino. El backend recibe las lecturas de los dos con el mismo código de registro, de modo que las mediciones son comparables, y una captura con tcpdump registra los bytes que circulan por ambos (Figura 2).

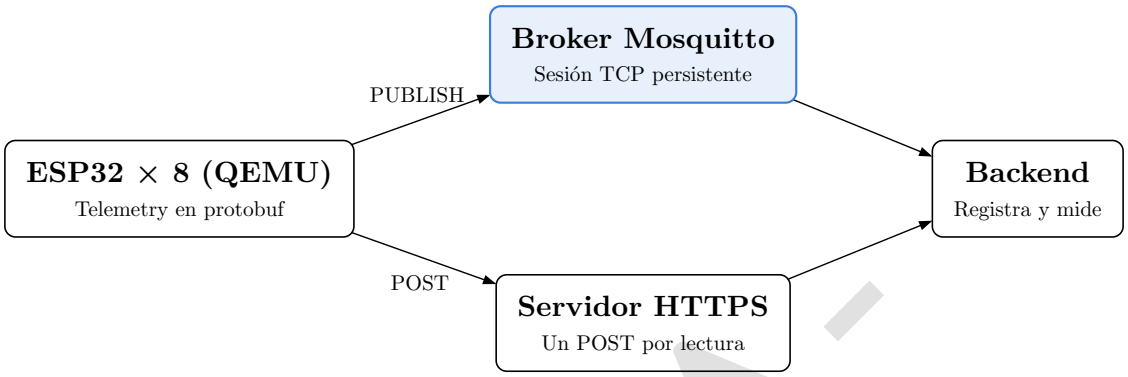


Figura 2: La misma lectura llega al backend por dos caminos; tcpdump captura los bytes de ambos.

Mensaje, tópicos y endpoints

El mismo contenido se envía en dos formatos. El campo repetido permite variar el tamaño del mensaje sin cambiar el esquema.

```
syntax = "proto3";

message Telemetry {
  uint32 esp_id          = 1;
  uint32 seq             = 2;
  uint64 t_us            = 3;
  uint32 status          = 4;
  repeated float values  = 5;
}
```

Su equivalente en JSON compacto, sin espacios, es:

```
{"esp_id":5,"seq":142,"t_us":1729690000000000,"status":0,"values":[23.4]}
```

La Tabla 5 indica el destino, el método y el cuerpo de cada combinación. En ambos casos `{esp}` es el número de nodo. En MQTT el formato se indica en el tópico, porque el protocolo no tiene un campo de tipo de contenido como el de HTTP, salvo a partir de MQTT 5.

Tabla 5: Destino y formato de cada camino.

Camino	Destino	Método	Cuerpo con Protobuf	Cuerpo con JSON
MQTT	Tópico `exp/tel/{esp}/pb` o `exp/tel/{esp}/json`	PUBLISH con QoS 0 o 1	`Telemetry` en binario	Texto JSON
HTTPS	Endpoint `/tel/{esp}`	POST	`Content-Type: application/x-protobuf`	`Content-Type: application/json`

Firmware

Emulación de la red

La pérdida y la latencia se inyectan con `tc netem` sobre la red Ethernet emulada.

Diseño de las corridas

El núcleo del experimento es una matriz de transporte por serialización: cuatro transportes por dos formatos dan ocho combinaciones (Tabla 6). Todas se corren con la configuración base: ocho ESP32, período de envío de 1 s, TLS activado, pérdida del 0 %, latencia de 0 ms y un valor por mensaje.

Tabla 6: Matriz de transporte por serialización.

Transporte	**JSON**	**Protobuf**
MQTT QoS 0	Combinación 1	Combinación 2
MQTT QoS 1	Combinación 3	Combinación 4
HTTPS reutilizado	Combinación 5	Combinación 6
HTTPS por request	Combinación 7	Combinación 8

Después se varía un factor a la vez (Tabla 7). Para no multiplicar las corridas, esas variaciones se aplican solo a MQTT con QoS 1 y a HTTPS con conexión reutilizada, cada uno con ambos formatos.

Tabla 7: Factores del experimento de telemetría.

Factor	**Valores a probar**	**Pregunta que responde**
Valores por mensaje	1, 10, 50	¿Cómo cambia el ahorro de Protobuf con el tamaño del mensaje?
Tipo de dato	Enteros chicos, floats, texto corto	¿Para qué datos ahorra más Protobuf?
Nodos ESP	2, 4, 8	¿Cómo escala la carga del servidor?
Período de envío	100 ms, 1 s, 10 s	¿Cuándo pesa el handshake?
TLS	Con y sin TLS	¿Qué parte del costo es cifrado?
Pérdida	0 %, 1 %, 5 %	¿Cómo se degrada la entrega?
Latencia	0 ms, 50 ms, 200 ms	¿Cómo afecta a cada handshake?

Cada configuración se repite cinco veces, y de las repeticiones se informan la media y el desvío. Cada corrida dura 120 s. El broker Mosquitto y el servidor HTTPS usan el mismo certificado de prueba, y una captura con tcpdump en la interfaz que usan las instancias registra los bytes de la corrida. Los mensajes, las métricas y la configuración usada se guardan en CSV o JSON. Los dos transportes nunca corren a la vez, para que no compitan por la CPU ni por la red.

Métricas

Los bytes se miden en la red y no en la aplicación, para incluir las cabeceras de TCP, de TLS y del protocolo (Tabla 8).

BORRADOR

Tabla 8: Métricas del experimento de telemetría.

Métrica	**Cómo se mide**	**Qué permite concluir**
Bytes por lectura	Captura con tcpdump: total de bytes / lecturas entregadas	Costo real de ancho de banda
Bytes de payload	Tamaño del cuerpo serializado, en JSON y en Protobuf	Ahorro de la serialización sola
Proporción payload / total	Bytes de payload / bytes por lectura	Cuánto pesa la serialización frente al transporte
Tiempo de serialización	Contador de ciclos en el firmware, al codificar y al decodificar	Costo de CPU de cada formato
Bytes de handshake	Bytes antes del primer dato en cada conexión	Peso de TCP y TLS
Paquetes por lectura	Captura con tcpdump	Overhead de acuses y respuestas
Latencia extremo a extremo	Recepción en el servidor — `t_us`	Rapidez de entrega
Tasa de entrega	Lecturas recibidas / enviadas, por `seq`	Efecto de la pérdida
Tiempo de conexión	Desde el SYN hasta el fin del handshake TLS	Costo de abrir conexiones
Factor de tiempo real	Tiempo simulado / tiempo del host	Validez de la corrida

El número de secuencia permite detectar pérdidas y también duplicados, que son esperables con MQTT QoS 1.

Validez

Además de las restricciones de la plataforma común, este experimento tiene las suyas:

- El handshake TLS en QEMU es más lento que en un ESP32 real, porque la criptografía se emula por software. Los tiempos de conexión con TLS se reportan como relativos, no absolutos.
- Solo se mide telemetría del nodo al servidor. El envío de comandos del servidor al nodo, donde HTTP necesita polling, queda fuera de esta comparación.
- El servidor HTTPS y el broker son implementaciones concretas; otras pueden dar números distintos.

1.3.3. Resultados

1.3.4. Análisis de resultados

BORRADOR

Conclusiones

BORRADOR

Bibliografía

[1] A. Banks, E. Briggs, K. Borgendale, y R. Gupta, Eds., «MQTT version 5.0», OASIS, OASIS Standard, 2019. [En línea]. Disponible en: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[2] P. T. Eugster, P. A. Felber, R. Guerraoui, y A.-M. Kermarrec, «The many faces of publish/subscribe», *ACM Computing Surveys*, vol. 35, n.º 2, pp. 114-131, 2003, doi: 10.1145/857076.857078.

[3] Google, «Overview». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://protobuf.dev/overview/

[4] L. Zhang, «Building Facebook Messenger». [En línea]. Disponible en: https://engineering.fb.com/2011/08/12/android/building-facebook-messenger/

[5] InOrbit, «InOrbit.AI: We make robots work». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://www.inorbit.ai/

[6] inorbit-ai, «edge-sdk-python: InOrbit Python Edge SDK». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://github.com/inorbit-ai/edge-sdk-python

[7] OpenRobOps, «OpenRobOps: Open source robot fleet management». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://openrobops.org/

[8] OpenRobOps/oro, «oro: OpenRobOps open-source robot fleet management and operations platform». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://github.com/OpenRobOps/oro

BORRADOR

Anexos

A. Mensajes del experimento de sincronización

Archivo `.proto` compartido por el firmware de los nodos y por el backend del experimento de sincronización de luciérnagas. Los tópicos por los que viaja cada mensaje se describen en la sección de mensajes y tópicos de ese experimento.

```
syntax = "proto3";

message Flash {
  uint32 esp_id = 1;
  uint32 led_id = 2;
  uint32 seq    = 3;
  uint64 t_us   = 4;
}

message PhaseReport {
  uint32 esp_id = 1;
  uint32 led_id = 2;
  float  phase  = 3;
  bool   on     = 4;
  uint64 t_us   = 5;
}

message RxEvent {
  uint32 esp_id  = 1;
  uint32 src_esp = 2;
  uint32 src_led = 3;
  uint32 seq     = 4;
  uint64 t_us    = 5;
}

message ExperimentConfig {
  uint32 num_esp      = 1;
  uint32 leds_per_esp = 2;
  float  epsilon      = 3;
  uint32 loss_pct     = 4;
  uint32 latency_ms   = 5;
  uint32 qos          = 6;
  uint32 seed         = 7;
}
```